NetSimX

Intelligent Network Routing & Traffic Simulator

Complete Project Documentation

Intelligent Network Routing & Traffic Simulator

From idea to working application:
what it is, why every file exists,
how it was built, and how to build something bigger on top of it

Version 1.0
July 2026

Table of Contents

How to Use This Document

This book is written for someone encountering NetSimX for the first time. Maybe you're inheriting this codebase from someone else, maybe you found the project and want to extend it, or maybe you're simply curious how a project like this actually comes together from a one-page idea into a working, tested application. No prior familiarity with the project is assumed anywhere in this document.

Networking concepts — routers, packets, TTL, Quality of Service, and so on — are explained in plain language the first time each one appears. A glossary near the end of this book collects all of them in one place if you need a quick refresher later, without hunting back through earlier chapters.

If you only have time to read one chapter, read Chapter 5, "How the Simulation Actually Works." Every other class, panel, and button in this project exists purely to serve that one loop — understand the loop, and the rest of the codebase reads as commentary on it.

> **A note on accuracy**
>
> Every screenshot in this book, and every specific figure quoted in the development-history and bug-report chapters, was captured or measured directly from the actual running application during development — not written from memory, and not aspirational. Where this book says something was verified, an automated check was genuinely run and its real output is what's reported here.

Chapter 1 — Origin Story: Why This Project Exists

NetSimX started from a written proposal for a Java-based Intelligent Network Routing and Traffic Simulator. The core problem it set out to solve was simple to state: computer networks are everywhere, but very few people ever get to see how a router actually decides where to send a packet, what happens when a link dies in the middle of a file transfer, or why a video call gets choppy while a background download keeps running fine. Real networking hardware is expensive and inconvenient to experiment on. Existing teaching simulators tend to fall into one of two traps — either they are too simplistic, amounting to little more than a static diagram with arrows drawn between boxes, or they are too opaque, producing a final summary number with no visibility into how that number was actually arrived at.

The proposal's stated objectives, restated here close to their original wording, became the specification for everything built afterward:

- Design a graph-based computer network simulator that the user builds themselves, rather than a fixed, pre-baked scenario.
- Simulate packet transmission between routers, hop by hop, and make that movement visible.
- Implement multiple real routing algorithms and allow the user to compare them directly.
- Simulate the messy realities of real networks: congestion, buffer overflow, packet loss, and link or router failures.
- Model the practical difference between TCP (reliable, acknowledged, automatically retried) and UDP (fire-and-forget) at the level of individual packets.
- Support Quality-of-Service prioritization, so that — for example — a voice-call packet can jump the queue ahead of a background file transfer during congestion.
- Recover automatically when part of the network goes down, exactly as a real routing protocol would.
- Optionally apply artificial intelligence — specifically reinforcement learning — to make smarter routing decisions than a fixed, static algorithm can.
- Present all of the above through a genuine, interactive graphical dashboard, not a scrolling log of numbers in a terminal window.

That list is the reason every module described later in this book exists. Nothing in NetSimX was designed in the abstract for its own sake; every class in the codebase maps back to one specific line in that original proposal. When this book later explains why a particular file exists, it is very often simply pointing back to one of these nine bullet points.

Chapter 2 — Technology Choices, and Why

Java 17 and JavaFX

The original proposal specifically called for a Java-based simulator with a real graphical interface, which settled the two biggest technology decisions immediately. JavaFX is the modern, actively maintained GUI toolkit for Java — the successor to the older Swing toolkit — and it came with everything this project needed already built in: a Canvas surface for drawing the network topology and animating packets by hand, chart components for the live performance graphs, and the full standard library of controls (buttons, tables, tabs, dialogs, menus) needed for everything else.

Java 21 was the original target version. Partway through development, however, a careful review of the codebase found that it did not actually use any language feature exclusive to Java 21 — nothing that would not compile perfectly well under the older, more widely installed Java 17. The project's minimum version requirement was lowered accordingly. This turned out to matter in practice, not just in theory: it is the difference between a new user being able to run the project with whatever recent Java installation they already happen to have, versus being forced to install an entirely separate JDK version before they can even attempt to build it. Chapter 6 tells the fuller story of exactly how this was discovered.

Maven

For anyone who has not used it before: Maven is a build tool. Rather than manually downloading library files and wiring them together by hand, you describe your project's dependencies — in this case, JavaFX for the interface and JUnit for automated testing — in a single file named pom.xml, and Maven takes care of downloading them and knows how to compile, test, and package the project with one command each: mvn compile, mvn test, and mvn package. This project's own pom.xml is deliberately short, because the list of things it actually depends on is deliberately short.

Almost no external dependencies

Beyond JavaFX for the interface and JUnit for testing — and JUnit is only needed to run the automated tests, not to run the application itself — NetSimX deliberately avoids pulling in additional libraries. Where a typical project of this size would reach for a JSON library, a PDF-generation library, or a database driver, this one has small, purpose-built replacements written by hand instead. This was a conscious trade-off rather than an oversight, and it is explained fully, with its costs and benefits laid out honestly, in Chapter 8.

Why fewer dependencies, specifically

Every external library a project depends on is also a promise that library will still exist, still be maintained, and still behave the same way years from now. For a project explicitly meant to be picked up and extended by someone else later, minimizing that surface area was judged more valuable than the convenience a library would have offered today.

Chapter 3 — Project Architecture

Before walking through every individual file in Chapter 4, it helps to see the shape of the whole system first. The diagram below was not drawn by hand — it was generated from a small Python script that lives alongside the project's other assets, specifically so it can be regenerated automatically if the package structure ever changes, rather than slowly drifting out of date the way hand-drawn architecture diagrams tend to.

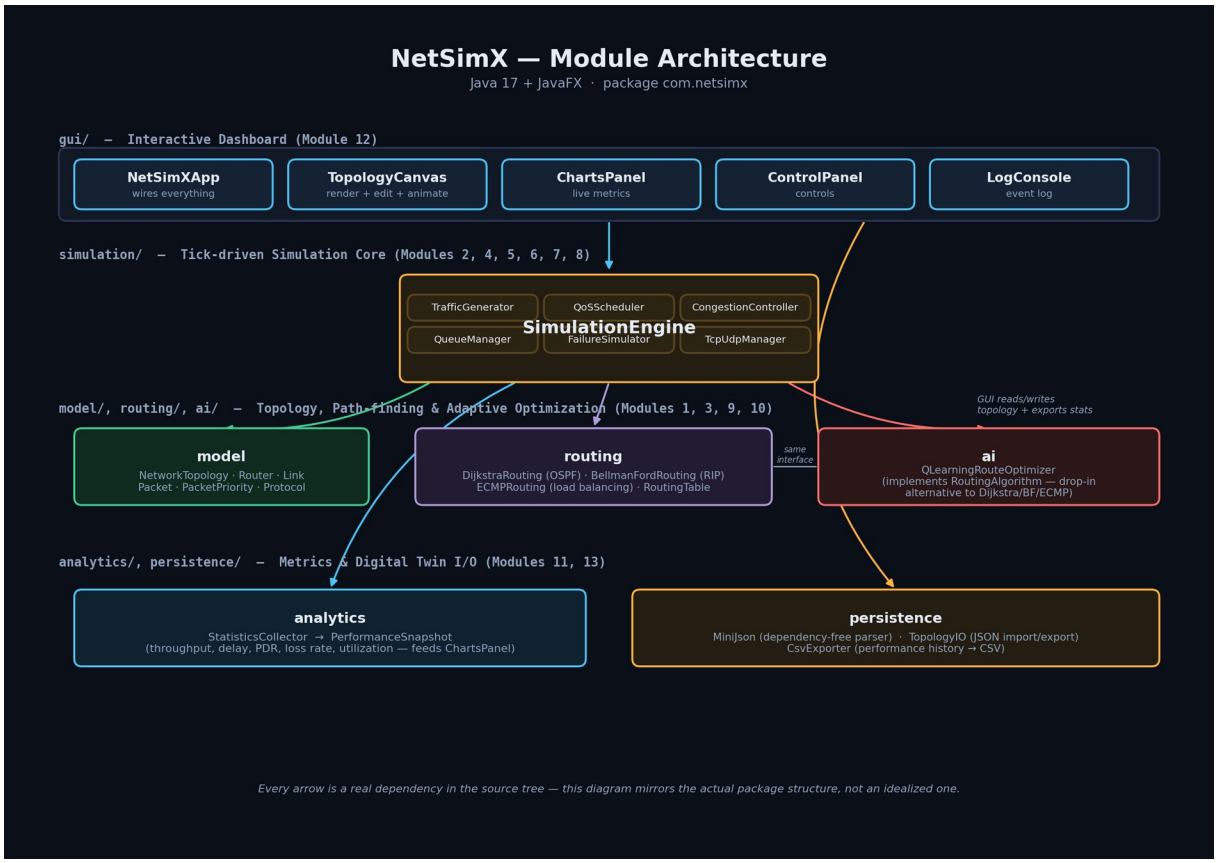

Figure 3.1 — The actual package dependency structure of NetSimX. Every arrow shown here is a real import relationship in the source code, not an idealized simplification.

Two things are worth noticing immediately in this diagram, because they explain a lot of design decisions found later in this book. First, the simulation package depends on model, routing, and ai, but nothing in those three packages depends back on simulation — the model classes have no idea a simulation engine even exists, and the routing algorithms have no idea whether they are being driven by a real interactive session or a headless benchmark run. Second, and most importantly, nothing in the entire project depends on the gui package. The simulation engine has no awareness that a graphical interface exists at all.

That second fact is precisely what makes it possible for BenchmarkRunner (introduced fully in Chapter 4) to run the exact same simulation engine used by the interactive dashboard thousands of times in a row, with zero windowing overhead, on a background thread, without the engine itself needing to know or care that nobody is watching.

Chapter 4 — The Project, File by File

This is the longest chapter in the book, and it is meant to be read the way you would explore the codebase yourself: folder by folder, in the order you would naturally encounter them. If you already know roughly what you are looking for, use the table of contents to jump directly to a package. If you are reading straight through, this chapter alone should leave you able to find your way around every corner of the source tree without getting lost.

```
netsimx/
├── pom.xml                    the build recipe (Maven)
├── .github/workflows/ci.yml   automatic build + test on every push
├── config/
│   ├── sample-network.json    the network loaded in demo mode
│   └── samples/               a small library of ready-made networks
├── docs/                      this book, plus every generated image
└── src/
    ├── main/java/com/netsimx/   all application source code
    └── test/java/com/netsimx/   all automated tests
```

4.1 — com.netsimx.model: the nouns of the simulation

This package defines the things that exist inside a simulated network. No real behavior lives here — these are data classes that every other package reads from and writes to. Think of this package as the vocabulary the rest of the project is written in.

Router.java

A single network node. Holds an identifier such as "R1", a human-readable label such as "Core-1" for display, an x/y position used purely for drawing it on the canvas, an up/down status, and — most importantly — a bounded queue of packets waiting to be forwarded onward. That queue has a fixed capacity. If it fills up, newly arriving packets are dropped. This single detail is what makes congestion a real, emergent behavior in this simulator rather than a number someone typed in by hand: if packets arrive at a router faster than it can send them onward, the queue genuinely fills up, and packets genuinely start being lost — exactly as happens on real hardware.

Link.java

A connection between two routers. Carries a cost (used by routing algorithms to identify the cheapest available path), a latency in milliseconds, and a bandwidth expressed in packets per second — how many packets it is physically able to carry during a single simulated tick. Two features were added to this class partway through development, both driven directly by the interactive dashboard's right-click menu: a lossProbability, an independent chance that a packet crossing this specific link is simply lost in transit, separate from ordinary congestion, modeling something like a noisy or damaged cable; and a pair of methods, congest() and releaseCongestion(), that let a user manually throttle a link's bandwidth on demand from the interface, to demonstrate backpressure and queue growth without needing to wait for organic traffic to build it up naturally.

Packet.java

A single unit of data traveling through the network. Carries a source router, a destination router, a size in bytes, a priority class, and a protocol (TCP or UDP). It also carries a TTL — Time To Live — a countdown that decreases by one every time the packet crosses a link; if that countdown reaches zero before the packet arrives at its destination, the packet is discarded. This is exactly the mechanism real IP networks use to guarantee a packet can never circulate forever if something goes wrong with routing. Packet also computes a checksum purely for the Packet Inspector panel's display — see the honest caveat about what this checksum does and does not currently represent in Chapter 8.

PacketPriority.java and Protocol.java

Two small enumerations. PacketPriority lists the five traffic classes in priority order — VOICE, VIDEO, WEB, EMAIL, FILE_TRANSFER — and that ordering is exactly what the QoS scheduler (Chapter 4.3) uses to decide which packet is sent first whenever a router cannot send everything at once. Protocol is simply TCP or UDP; the behavioral difference between the two — automatic retries, acknowledgements — deliberately does not live in this enum at all, it lives in simulation.TcpUdpManager, keeping this package limited to plain data.

NetworkTopology.java

The container that holds every router and every link belonging to one network, along with fast lookups — which links touch a given router, which neighboring routers are currently reachable given the present up/down state of everything around them. Every other package in the project reads and writes a NetworkTopology rather than manipulating routers and links directly; it is the single source of truth for "what does the network currently look like."

4.2 — com.netsimx.routing: deciding where a packet goes next

Everything in this package answers one question: given a starting router, what is the best path to every other router in the network? Four different, genuinely distinct answers to that question are implemented, all behind one shared interface.

RoutingAlgorithm.java

An interface with a single method: given the current network topology and a source router, compute the best path to every other router. This one interface is the seam that makes it possible to swap algorithms live from a dropdown menu in the dashboard without any other part of the application needing to change. Every algorithm described below implements exactly this interface, and nothing else in the codebase cares which one is currently active.

DijkstraRouting.java

Dijkstra's algorithm — the textbook shortest-path algorithm most computer science students encounter early on — modeling OSPF, a real-world routing protocol used inside large networks. Every router is assumed to know the entire network map in advance and picks the mathematically cheapest path to any destination.

BellmanFordRouting.java

A different shortest-path algorithm, modeling RIP, another real-world protocol. Unlike Dijkstra, it does not require full network visibility up front — in a real network it converges by routers repeatedly gossiping distance information to their immediate neighbors. For a single, static snapshot of a topology, it mathematically produces the same shortest-path answer as Dijkstra — there is an automated test, dijkstraAndBellmanFordAgreeOnCost, that checks exactly this — but it represents a genuinely different real-world approach to arriving at that same answer.

ECMPRouting.java

Equal-Cost Multi-Path routing. Whenever multiple paths of the exact same cost exist between two points, this algorithm spreads traffic across all of them in round-robin fashion instead of always favoring a single one. This is what makes load balancing a real, observable effect rather than a cosmetic label: the choice of path is recomputed live, on a per-packet basis, so two packets sent moments apart between the same two routers can genuinely travel by different physical routes.

RoutingTable.java

Caches whatever routes the currently active algorithm has computed, and knows precisely when those routes need to be recomputed — any time the topology itself changes, such as a link failing or a new router being added. This same cached data is also exactly what powers the Routing Table inspector panel described later in this chapter, so what you see on screen when you click a router is not a separate calculation, it is a direct view onto this class's internal state.

4.3 — com.netsimx.simulation: the engine room

This is the largest and most important package in the entire project. Everything here cooperates, once per simulated tick, to move the whole network state forward by one step. SimulationEngine is the conductor; everything else in this package is an instrument it plays in a fixed order. Chapter 5 walks through that order in full narrative detail — this section instead introduces each individual class.

SimulationEngine.java

Owns the main loop. A single call to its tick() method performs five distinct steps every time it runs: possibly inject a random failure if chaos mode is enabled, generate any new traffic due this tick, check whether any TCP packets have been waiting too long for an acknowledgement, forward every router's queued packets one hop each, and finally record a fresh set of statistics. Chapter 5 is dedicated entirely to walking through exactly what happens inside each of those five steps.

TrafficGenerator.java

Responsible for creating new packets in the first place. Built around five realistic traffic profiles — Voice, Video, Web, Email, File Transfer — each with a believable packet size range, priority level, and protocol choice. Voice traffic, for instance, is modeled as small, frequent, high-priority, and UDP, which mirrors how real voice-over-IP systems actually behave: a voice packet that arrives late is essentially useless, so nothing is gained by retransmitting it, and UDP's lack of retransmission overhead is a genuine advantage rather than a shortcut. File Transfer, by contrast, is modeled as large, low-priority, and TCP, because correctness — every byte arriving intact — matters more than raw speed for a file download.

QoSScheduler.java

Whenever a router has more packets ready to send in a given tick than its outgoing link can actually carry, this class decides which ones are sent first, in strict priority order: Voice, then Video, then Web, then Email, then File Transfer. This is the class that turns "Quality of Service" from a checkbox in a feature list into a real, visibly different outcome for different kinds of traffic under load.

CongestionController.java

Converts a link's configured bandwidth figure into a concrete answer to the question "how many packets can actually cross this specific link during this specific tick," and separately tracks a rolling utilization percentage for every link that gradually decays back toward zero whenever that link goes idle. This is the class directly responsible for the utilization numbers and the color-coded link thickness you see change in real time on the dashboard.

QueueManager.java

Tracks every router's queue occupancy over time, including its historical high-water mark, and — added specifically to support the Report screen introduced in Chapter 4.9 — counts distinct congestion episodes rather than every individual tick spent congested. A fifty-tick traffic jam is counted as one event, not fifty, because the counter only increments at the precise moment a router transitions from not-congested to congested, not on every tick it happens to remain congested afterward.

FailureSimulator.java

Responsible for taking a link or router down, and bringing it back up again — whether triggered manually by right-clicking something in the dashboard, or automatically and at random when chaos mode is switched on. Also counts the total number of failure events that have occurred, again feeding directly into the Report screen.

TcpUdpManager.java

Models the genuine behavioral difference between TCP and UDP described back in Chapter 1. UDP packets are truly fire-and-forget — no code in this file ever touches a UDP packet at all. TCP packets, by contrast, are tracked from the moment they are sent: if one is dropped, or if its acknowledgement simply does not arrive within a configured timeout window, it is automatically retransmitted, up to a maximum number of attempts, after which the connection gives up entirely, mirroring how a real TCP connection eventually times out for good. This particular file is also the subject of an entire chapter of its own — two real, previously-shipping bugs were discovered inside it during development, and Chapter 7 walks through both in full, precisely because how they were found is at least as instructive as what they were.

BenchmarkRunner.java

Added specifically to support Benchmark Mode. Runs a given topology-and-traffic scenario headlessly — no graphical interface, and no waiting for real wall-clock time to pass; tick() is simply called thousands of times back to back, as fast as the processor allows — once per selected routing algorithm, repeated multiple times each, and averages the resulting figures so that different algorithms can be compared fairly under identical conditions rather than under whatever traffic happened to occur during one lucky or unlucky run.

4.4 — com.netsimx.ai: the adaptive route optimizer

QLearningRouteOptimizer.java

A genuine, fully working reinforcement-learning agent, implemented entirely from scratch in plain Java with no external machine-learning library involved. For readers who have not encountered Q-learning before: the agent maintains a table recording, roughly, "how good has it historically turned out to be, when trying to reach destination Y, to take the next hop from router X toward neighbor Z." Every single time it actually attempts a hop, it observes what happened — was that link congested, was there packet loss — and nudges the corresponding entry in its table up or down accordingly. Given enough time and enough attempts, it gradually learns to route around congestion without ever being explicitly told any rule of routing at all; it learns purely from repeated experience, the same way the algorithm's name suggests. It implements the identical RoutingAlgorithm interface every other algorithm in Chapter 4.2 implements, so the dashboard is able to swap it in exactly the same way it swaps in Dijkstra or ECMP. Chapter 8 explains, honestly, why this is a from-scratch Java implementation rather than the Python and Stable-Baselines3 combination originally proposed.

4.5 — com.netsimx.analytics: turning raw numbers into a story

StatisticsCollector.java and PerformanceSnapshot.java

StatisticsCollector records what happened during every tick — packets generated, delivered, dropped — and periodically produces a PerformanceSnapshot: a single point-in-time bundle capturing throughput, average delay, packet delivery ratio, loss rate, and utilization all together. This is precisely the data structure that feeds every one of the dashboard's live charts, and it also knows how to format itself as a single row of a CSV file, which is what the Export Stats CSV button and the Report screen's CSV download both ultimately rely on.

4.6 — com.netsimx.persistence: saving and loading things

Every file in this package deliberately avoids external libraries, for the reasons discussed fully in Chapter 8.

MiniJson.java

A small but complete JSON reader and writer, written entirely by hand rather than pulled in from a library. It is used throughout the project to save and load network topologies as ordinary .json files that are easy to open, read, and hand-edit in any text editor.

TopologyIO.java

Uses MiniJson to convert a NetworkTopology to and from a JSON file on disk. This is the class directly behind the Load JSON and Save JSON buttons in the dashboard's sidebar.

CsvExporter.java

Writes the collected performance history out to a .csv file, ready to open directly in Excel, Google Sheets, or any other tool that reads comma-separated values.

MiniPdf.java

A hand-written PDF file generator. This may sound surprising, but PDF is, underneath its file extension, a largely plain-text format — and for a document that is simply a title followed by lines of text, it is entirely possible to generate a fully valid PDF file without depending on any library at all. This class is exactly what the Report screen's Download PDF button uses. It was verified during development by actually generating a sample PDF with it and then extracting the text back out using an independent external tool, pdftotext, to confirm a real PDF reader can genuinely open and correctly parse the result — not merely that the code ran without throwing an error.

RecentProjects.java

Remembers which topology files have recently been opened, persisting that short list to a small file in the user's home folder, at ~/.netsimx/recent-projects.json, so the Home Dashboard's Recent Projects list survives being closed and reopened.

4.7 — com.netsimx.topology: building networks automatically

TopologyGenerator.java

Added specifically to support the New Simulation Wizard. Rather than forcing every user to place each router and link by hand before they can run anything, this class can generate a Star (one central hub connected to many spokes), a Mesh (every router connected directly to every other router), a Ring (a single closed loop), a Tree (a branching two-level hierarchy), or the same ISP Backbone shape used by the bundled sample network — each with sensible default costs, bandwidths, and on-screen positions, so a freshly generated network already looks reasonable the instant it appears rather than needing to be manually rearranged.

Chapter 4 (continued) — Everything You See and Click

The gui package is organized as one class per screen or panel, so each individual piece of the interface can be read and modified on its own without needing to understand the entire package at once. The screenshots in this section were all captured directly from the running application during development.

NetSimXApp.java — the entry point

Owns the single application window and the single Scene inside it; every screen change described in this section — Splash to Wizard, Wizard to Workspace, Workspace to Benchmark, and so on — is implemented as simply swapping what content currently sits inside that one Scene. It also owns the two "clocks" that make the simulation move on screen: a Timeline that calls engine.tick() on a fixed schedule, and a separate AnimationTimer that smoothly interpolates each packet's dot between ticks, so motion on the canvas reads as fluid movement rather than packets instantly teleporting from router to router.

Launcher.java — a small but important workaround

An almost-empty class whose entire purpose is working around one specific, well-known Java packaging quirk: if a runnable jar file's declared main class directly extends javafx.application.Application, running it with the plain command java -jar yourapp.jar fails outright with the error "JavaFX runtime components are missing." Java refuses to start it that way. Routing the jar's entry point through this small indirection class instead — which does not itself extend Application, it simply calls NetSimXApp.main() — sidesteps that restriction entirely. This is a standard, widely known pattern in the Java community rather than something unique to this project, but it is exactly the kind of detail that silently breaks a downloaded project for anyone unaware of the trick, and it was in fact discovered as a genuine bug during this project's own development — see Chapter 6.

SplashScreen.java

The very first screen shown when the application launches. Four buttons: New Simulation, Open Project, Benchmark Mode, and Documentation.

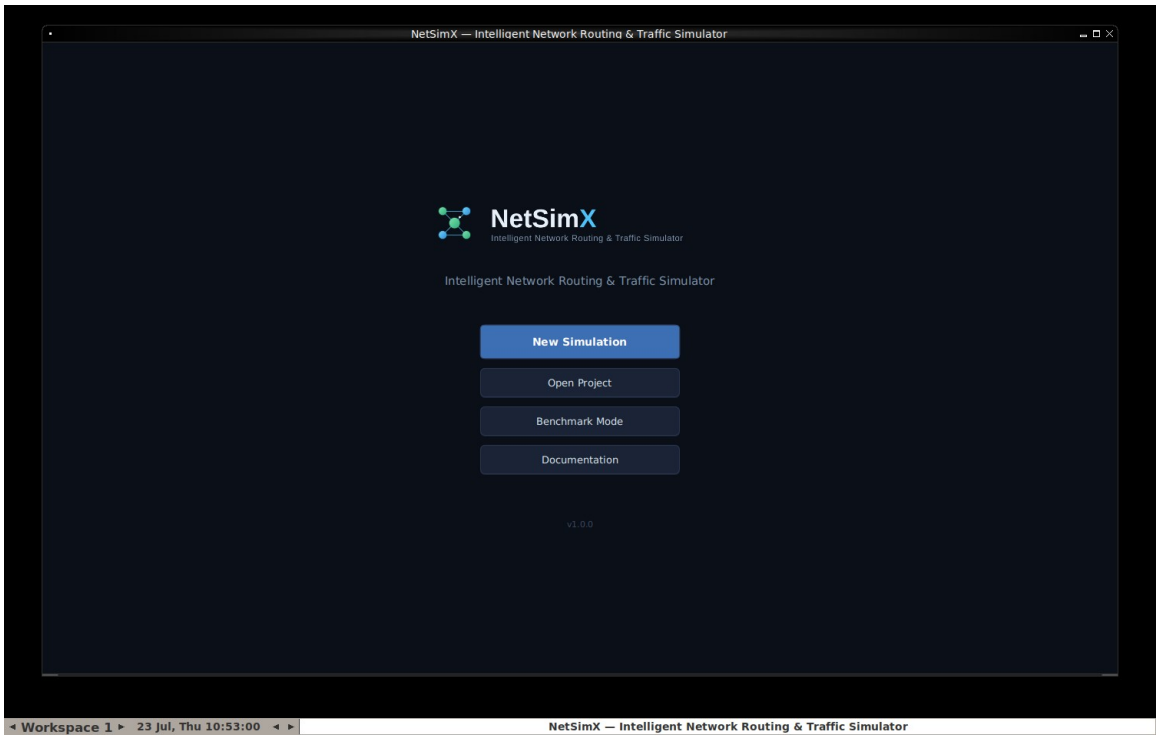

Figure 4.1 — The Splash Screen, captured from the running application.

NewSimulationWizard.java

The form a user fills out to start a fresh simulation: a name, a topology template chosen from the generators described in section 4.7, a starting routing algorithm, and a traffic preset. Clicking Create hands all four choices to NetSimXApp, which builds the corresponding network and drops the user straight into a live, already-running workspace.

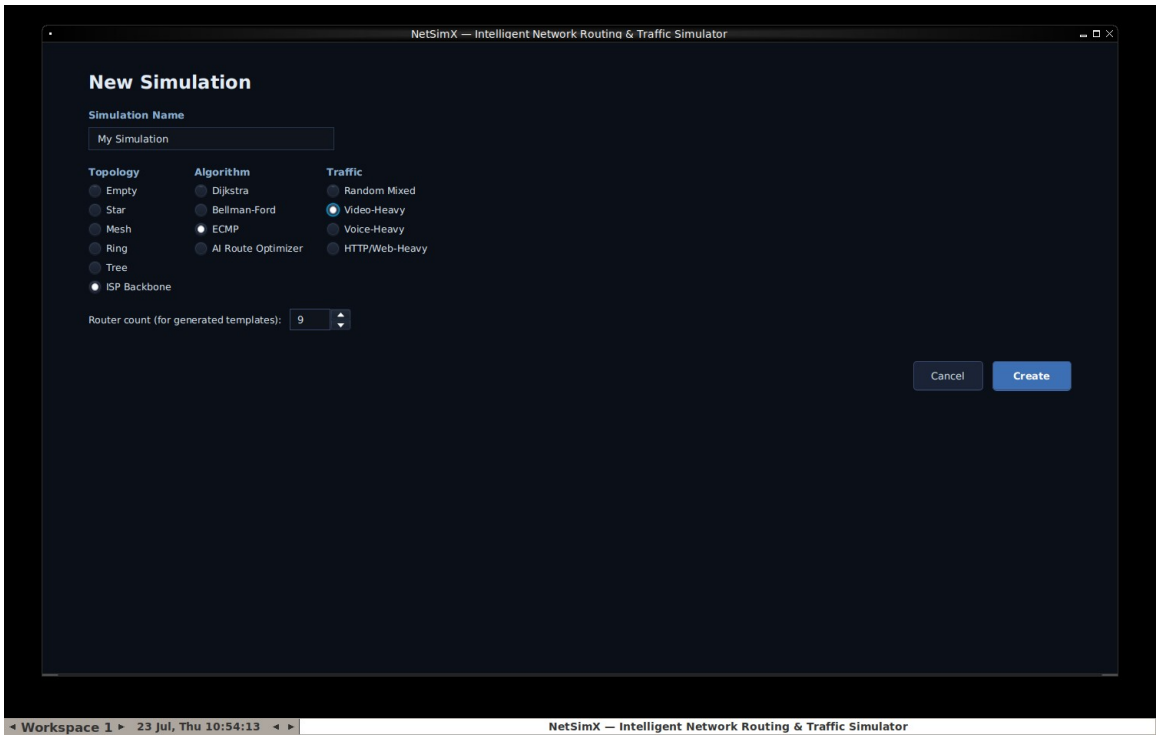

Figure 4.2 — The New Simulation Wizard, with ISP Backbone, ECMP, and Video-Heavy selected.

HomeDashboard.java

A lighter-weight landing page, reachable from inside a running workspace via its new Home button. Shows the Recent Projects list described in section 4.6, alongside quick actions for starting a new simulation, importing a JSON topology, jumping to Benchmark Mode, or browsing the bundled sample topology library.

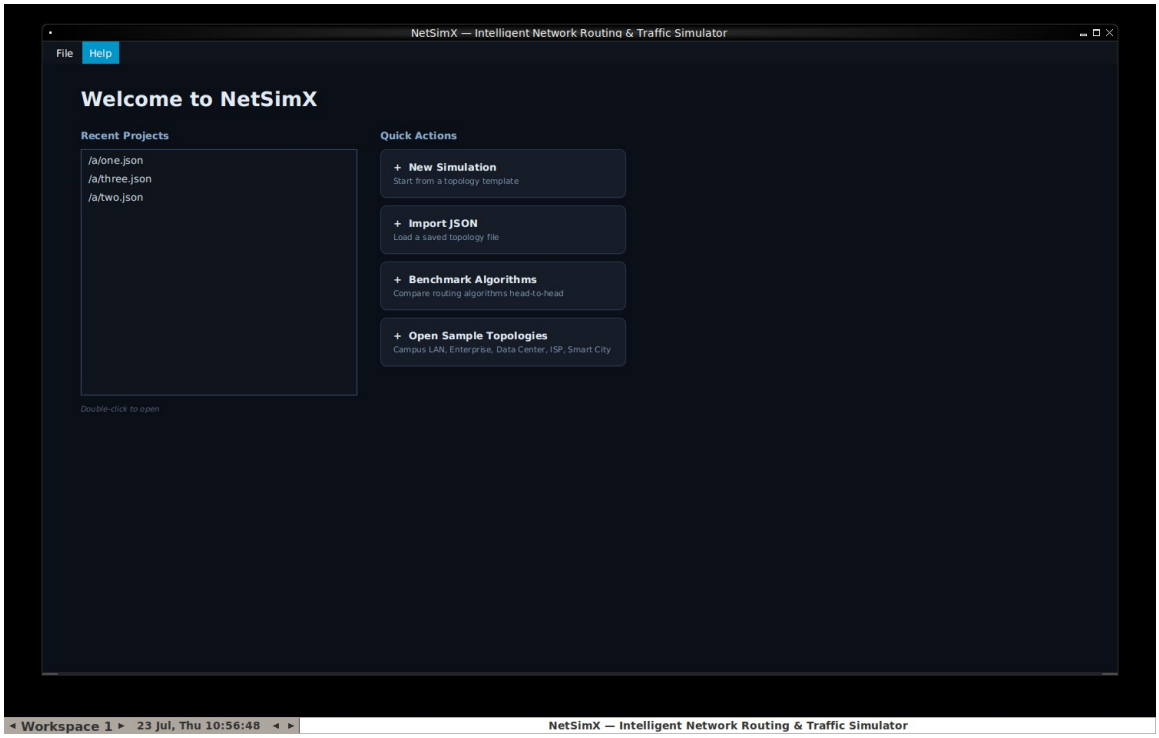

Figure 4.3 — The Home Dashboard, showing Recent Projects and Quick Actions.

TopologyCanvas.java — the heart of the visual experience

A raw JavaFX Canvas, meaning nothing on it is a pre-built interface widget — every router circle, every link line, and every animated packet dot is drawn by hand using low-level drawing instructions such as "fill this circle at these coordinates" or "stroke a line between these two points." This class also handles every form of mouse interaction available on the network diagram itself: dragging a router to reposition it, clicking to select a router, a link, or an in-flight packet, and right-clicking to open a context menu.

ControlPanel.java

The left-hand sidebar visible throughout the workspace: simulation start, pause, step, and reset controls; the routing algorithm selector; simulation speed; topology editing tools; traffic generator controls; and failure and chaos-mode controls.

The three inspector panels

NetworkPanel.java, PacketInspectorPanel.java, and RoutingTablePanel.java together form the three tabs on the right-hand side of the workspace. Clicking any router shows its live status, queue occupancy, and neighbor list in NetworkPanel, or its complete live routing table in RoutingTablePanel. Clicking any moving packet dot directly on the canvas opens PacketInspectorPanel, showing that specific packet's source,

destination, current router, remaining TTL, delay so far, and more, updating live for as long as that packet remains in flight.

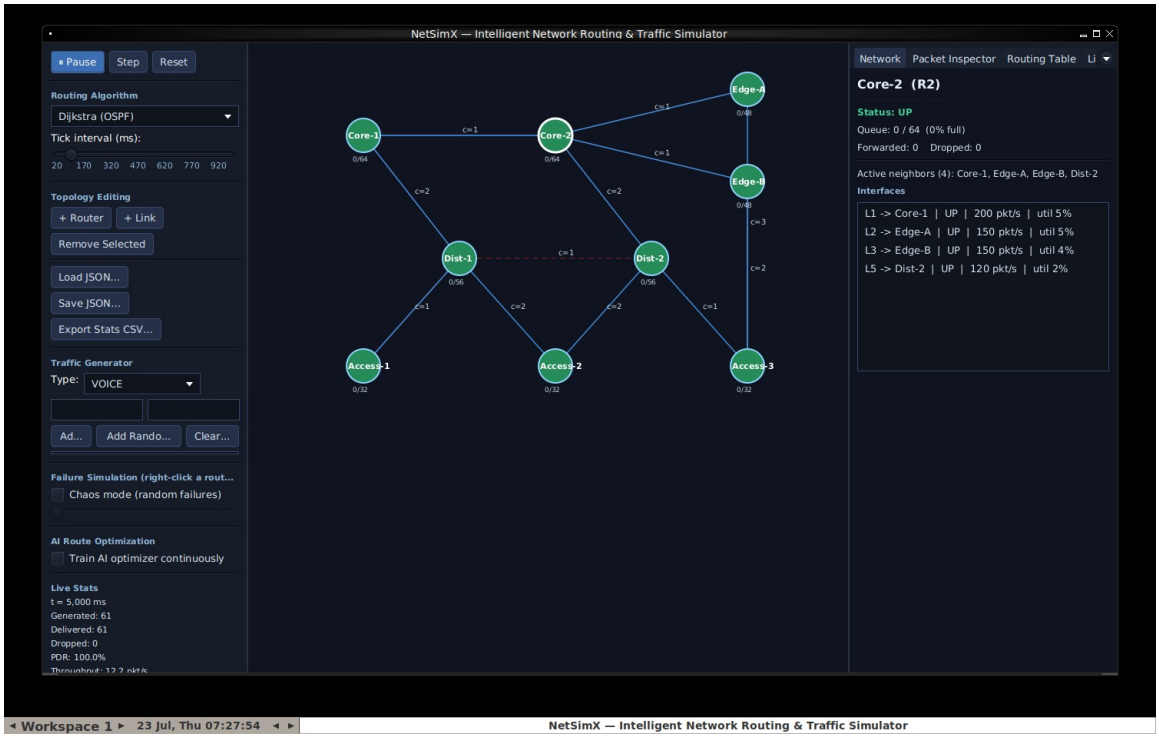

Figure 4.4 — The Network inspector panel, showing a selected router's live interface detail.

ChartsPanel.java and LogConsole.java

ChartsPanel draws four live line charts — throughput, average delay, packet delivery ratio, and utilization — each fed one new data point every simulated tick. LogConsole is a scrolling text log recording every notable event as it happens: drops, retransmissions, failures, and recoveries.

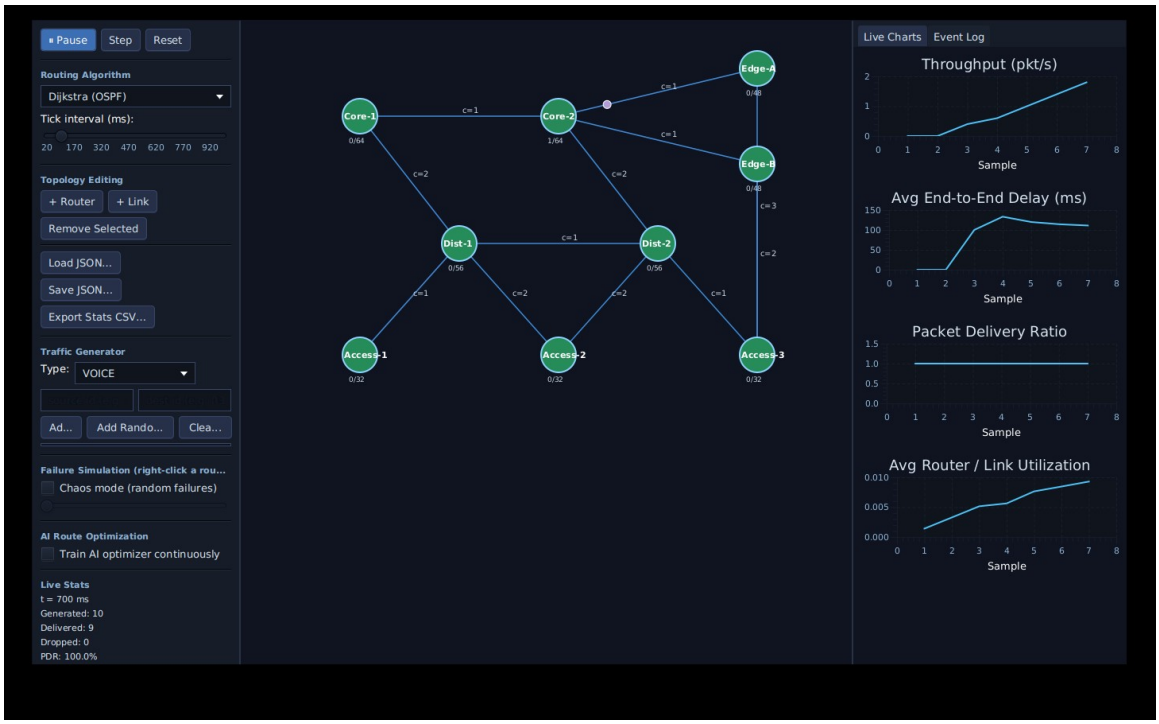

Figure 4.5 — The full workspace while a simulation is actively running, with traffic flowing and charts populating live.

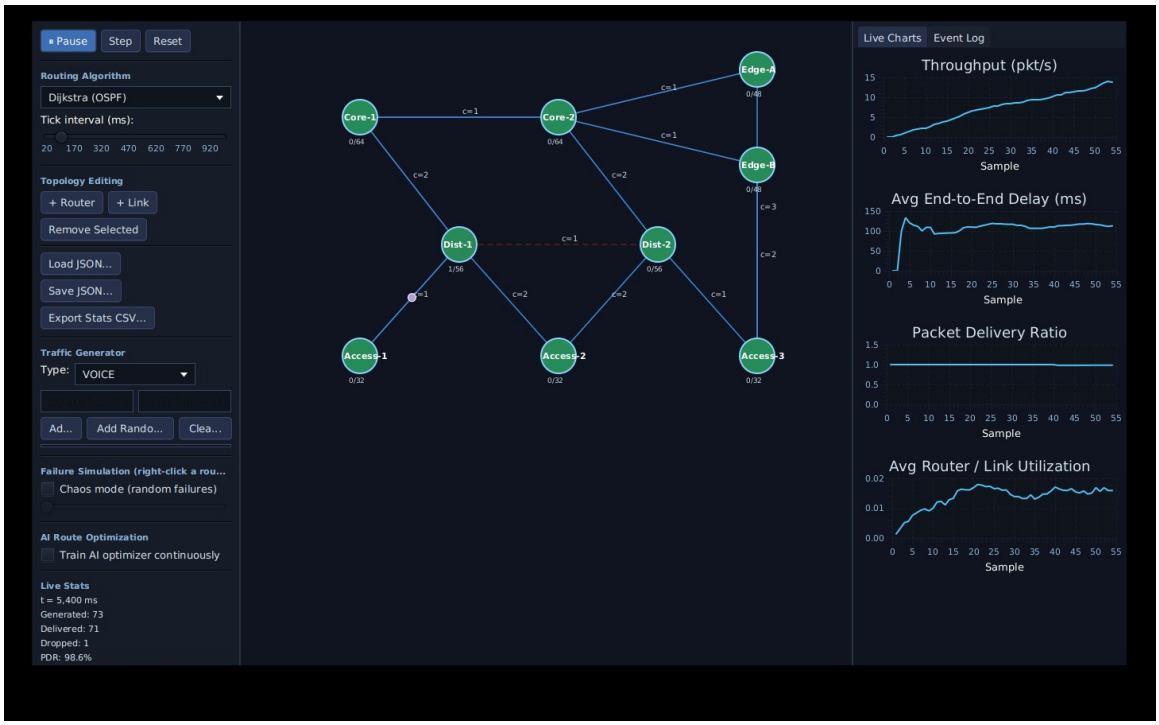

Figure 4.6 — A link has just been failed (shown in dashed red); the routing engine has already recomputed and rerouted traffic around it.

BenchmarkScreen.java

Lets the user pick which algorithms to compare, how many independent runs to average per algorithm, and how many simulated ticks each run lasts, then presents a results table and a bar chart once the comparison finishes running on a background thread.

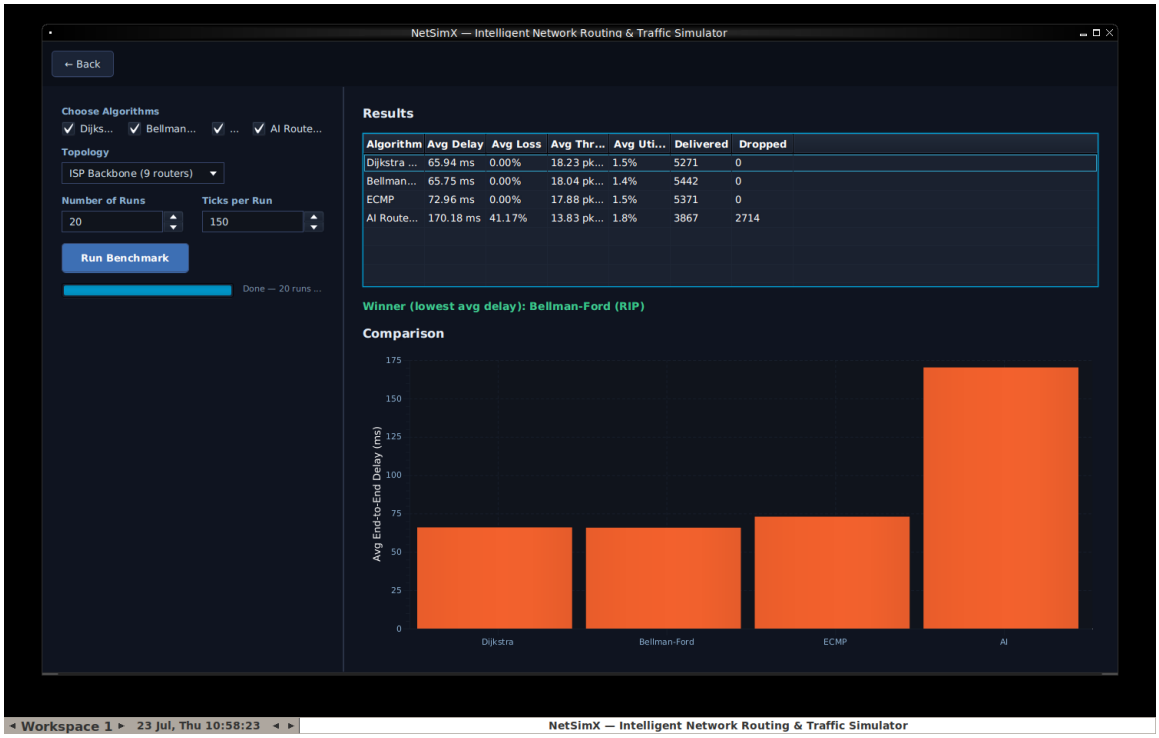

Figure 4.7 — Benchmark Mode after a completed run, comparing four algorithms with a computed winner.

ReportScreen.java

Presents a plain-language summary of the current simulation session — topology, algorithm, simulation time, packets generated and delivered, loss rate, average delay, bandwidth utilization, congestion events, failures, TCP retransmissions, and, if the AI optimizer is active, its training step count — with Download PDF and Download CSV buttons.

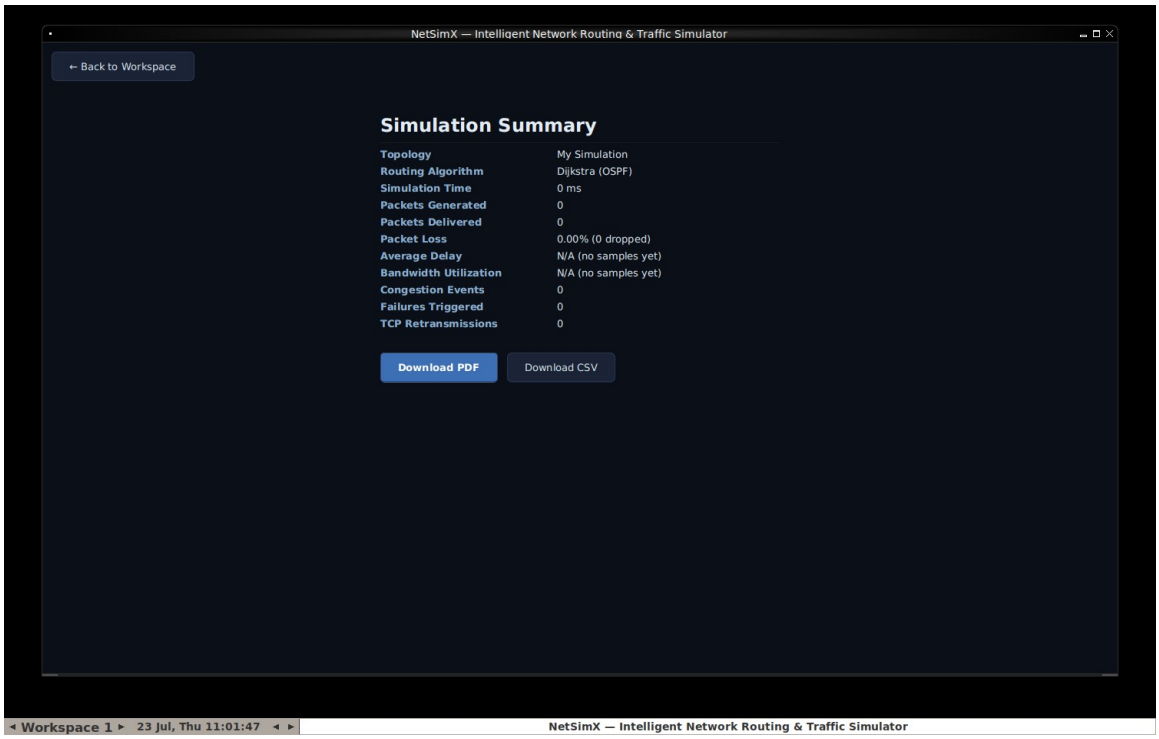

Figure 4.8 — The Report Screen, summarizing a simulation session.

4.8 — config/: starter data

sample-network.json is the nine-router "Campus LAN" network automatically loaded whenever the application is launched in demo or recording mode. The samples/ folder holds five ready-made networks — Campus LAN, Enterprise Network, Data Center, ISP Backbone, and Smart City IoT — offered from the Home Dashboard and the wizard; some were built by hand and some were generated with TopologyGenerator and saved out to JSON for reuse.

4.9 — src/test/java: the automated safety net

Every package described above that contains real logic, rather than purely visual interface code, has a matching JUnit 5 test class: RoutingAlgorithmsTest, SimulationEngineTest, LinkMechanicsTest, TcpUdpManagerTest, TopologyGeneratorTest, BenchmarkRunnerTest, RecentProjectsTest, and MiniPdfTest — thirty-eight tests in total, all passing, run automatically on every single push to the project's repository via the GitHub Actions workflow defined in .github/workflows/ci.yml. ManualSmokeCheck.java is a separate, deliberately simpler end-to-end check that can be run directly with the plain java command, without needing Maven or JUnit installed at all, intended as a fast first sanity check.

Chapter 5 — How the Simulation Actually Works

This is the chapter to read carefully if the goal is to genuinely understand this project, rather than merely knowing where each file lives. Everything else described in this book is scaffolding built around the single loop explained here.

Once a simulation is running, SimulationEngine.tick() fires on a timer — by default, once every one hundred simulated milliseconds — and every single time it fires, it performs exactly five steps, always in the same order.

Step 1 — Maybe break something

If chaos mode has been switched on in the control panel, there is a small, configurable random chance that a link is taken down during this specific tick. This single step is what turns failure-and-recovery behavior from something a user has to manually trigger by right-clicking, into an ongoing, unattended stress test that keeps the network honest for as long as the simulation keeps running.

Step 2 — Make new packets

Every currently configured traffic flow rolls a probability check this tick, based on its own configured rate. If that check succeeds, a brand-new Packet object is created, a route for it is computed immediately using whichever routing algorithm is presently active, and the packet is placed into its source router's outgoing queue. If no route to the destination currently exists at all, the packet is dropped immediately, and that drop is recorded as a genuine loss.

Step 3 — Check for stuck TCP packets

Any TCP packet that has been sitting and waiting for its acknowledgement for longer than the configured timeout window is automatically resent at this point, exactly as a real TCP implementation would do. This is also, not coincidentally, the exact mechanism whose implementation contained the two bugs documented in full in Chapter 7.

Step 4 — Move everything one hop

This is by far the busiest of the five steps, and it happens once for every router that currently has packets waiting in its queue. First, the engine works out exactly how many packets that router is physically permitted to send onward this tick, based on the combined bandwidth of its outgoing links. Next, the QoS scheduler is asked to pick which of the waiting packets go out first, strictly in priority order. Then, for each packet chosen to move this tick: a check is rolled for random line-error loss on that specific link, the packet's TTL is decremented by one, and the packet is either delivered, dropped, or simply left in its current queue for another attempt next tick.

Step 5 — Take a snapshot

Finally, every router's current queue occupancy is recorded, and a complete performance snapshot — throughput, average delay, loss rate, and utilization, all as of this exact moment — is generated. This is precisely the data that feeds the dashboard's live charts, and it is also exactly the same data eventually summarized on the Report screen described in Chapter 4.

The whole point of this chapter

Every visible feature in the dashboard is simply a different visual window onto this exact same five-step loop running over and over, many times a second. There is no separate code path for what the user sees versus what actually happens; the dashboard is a live view of the real internal state, nothing more and nothing less.

Chapter 6 — The Development Journey

This chapter is a chronological account of how NetSimX was actually built, in the order the work genuinely happened, including the mistakes and how they were fixed — because that is usually the most useful part of a "how was this built" account for anyone trying to learn from it.

Phase 0 — the initial build

Starting directly from the original written proposal described in Chapter 1, the entire application was scaffolded in one continuous pass: the Maven project itself, every model, routing, simulation, AI, analytics, and persistence class, and a first working version of the JavaFX dashboard — all compiled, covered by automated tests, and actually launched and screenshotted before being considered finished, rather than simply assumed to work because the code looked correct. The AI module and the persistence layer were both deliberately adapted away from the original proposal's specific technology suggestions at this very first stage, for the reasons given in full in Chapter 8.

Fixing "why won't it even run"

The very first real obstacle encountered was not a bug in the application at all — it was simply getting a working Java installation and Maven properly available on a Windows machine's system PATH, plus discovering along the way that the project had been configured to require Java 21 when it did not, in fact, use anything specific to Java 21 anywhere in its code. The minimum version requirement was lowered to Java 17 as soon as this was noticed, removing an unnecessary barrier for anyone else trying to build the project for the first time.

"The window isn't adjusted properly"

Early in development, the application's window could open partially off-screen depending on a user's specific monitor configuration, because its size and position had simply been hardcoded to fixed pixel values. The fix was to query the actual usable screen area at startup and size and center the window against that real, measured value instead of an assumption — and, importantly, this fix was verified by deliberately testing it against a much smaller virtual screen than the original development machine used, not merely re-tested on the same screen the bug had originally been noticed on.

Building out a professional presentation

A substantial effort went into making the project look and read like a genuinely polished, professional open-source project: a proper README file with status badges, a hand-designed logo built as a scalable vector graphic and then rendered down to PNG images, a real architecture diagram generated from a small script rather than drawn by hand, a scripted demo mode that reliably drives the application through a realistic scenario purely for the purpose of recording screenshots and a demonstration video, and a continuous-integration workflow so that a "build passing" badge on the project actually means something verifiable rather than being purely decorative. While specifically testing the instructions for downloading and directly running the packaged application jar, a genuine bug was discovered — the "JavaFX runtime

components are missing" failure described in Chapter 4 — which is exactly why the small Launcher.java workaround class exists today.

Inspector panels and context menus

The dashboard gained three new click-to-inspect panels and full right-click context menus on both routers and links. Verifying this phase of work surfaced an interesting problem with the testing environment itself, rather than with the application: JavaFX right-click context menus render as separate pop-up windows at the operating-system level, and the particular headless testing environment being used during development did not have a window manager running, which silently prevented those pop-up windows from displaying correctly during automated screenshot testing. Installing a minimal window manager purely for testing purposes — with no change whatsoever to the application's own code — resolved the issue immediately, and the underlying context-menu implementation turned out to have been entirely correct the whole time.

App navigation

The Splash Screen to Home Dashboard to New Simulation Wizard to Workspace flow described throughout Chapter 4 was added during this phase, along with the procedural topology generators and the small library of sample networks.

Benchmark Mode and Reports

The headless multi-run benchmark comparison and the PDF and CSV report export features were the last major pieces added. Building the benchmark runner — which, by its very nature, exercises long-running TCP sessions far more heavily and for far longer than a few minutes of manually clicking around the interactive dashboard ever realistically would — is precisely what surfaced two genuine, previously-shipping bugs in the TCP retry logic. Both are documented in complete detail in the next chapter, because the process of how they were found is, if anything, more instructive than the bugs themselves.

Chapter 7 — Bugs Found and Fixed: A Case Study

Two genuine bugs were discovered in TcpUdpManager while stress-testing the newly built benchmark runner. Neither bug was ever visible during ordinary interactive use of the dashboard — both required a TCP flow to run for long enough, with enough simultaneous activity, to reach an edge case that a few minutes of manual clicking simply does not reliably reach. This chapter exists because it is a good, concrete illustration of why automated, high-volume testing matters even for a project that already appears, on the surface, to be working perfectly well.

Bug 1 — a crash

TcpUdpManager.checkTimeouts() loops over a list of every packet currently waiting for an acknowledgement, looking for any that have waited too long. For each one it finds, it needs to create a retransmitted copy and add that new copy to the very same waiting-list it is presently looping over. The original code performed that addition while still actively inside the loop — something Java explicitly forbids, throwing a ConcurrentModificationException specifically because modifying a collection while iterating over it can silently corrupt the iteration in ways considerably worse than a clean, obvious crash. This particular failure only occurred when two or more packets happened to time out within the exact same simulated tick — rare enough during casual, everyday use of the dashboard, but something the benchmark runner's sustained, high-volume traffic hits constantly and reliably. The fix was to collect every needed change while looping, without touching the underlying list, and only apply all of those collected changes afterward, once the loop has fully finished.

Bug 2 — a silent correctness bug

This second bug is arguably worse than the first, precisely because nothing about it ever announces itself — there is no crash, no error message, nothing in a log file to notice. Every TCP packet is supposed to give up retrying and stop entirely after a configurable maximum number of attempts. The counting logic that tracks how many attempts a given packet has made was, in isolation, completely correct. But entirely separately, every single time any packet — whether freshly created or a retransmission — is handed off to the network, a different and seemingly unrelated method, onTcpPacketSent, unconditionally reset that packet's attempt counter straight back down to one. Because every retransmitted packet naturally does get handed back to the network as an ordinary part of normal delivery, its carefully incremented attempt count was being silently erased every single time this happened — meaning the "give up after N attempts" safety limit never actually functioned as intended. In principle, a sufficiently unlucky TCP flow could have retried forever, without any visible indication that anything was wrong. The eventual fix was a genuinely tiny, one-word change — replacing a plain map insertion (put) with a conditional one (putIfAbsent), so that a retry's already-correct, already-incremented attempt count is preserved rather than blindly overwritten — but actually finding this bug required deliberately writing a test that reproduced the real sequence of calls the engine genuinely makes during normal operation, rather than testing the class's methods in isolation from how they are actually used together in practice.

The lesson worth carrying forward

Both fixes now have permanent, dedicated regression tests, specifically so neither bug can silently reappear later without a failing test immediately calling attention to it. If you extend this project, writing a test that reproduces the real sequence of calls your code will actually experience in production — not just calling a method once in isolation — is consistently what catches the bugs that matter most.

Chapter 8 — Design Decisions and Honest Trade-offs

Every real project accumulates places where the theoretically ideal implementation was not practical given the time and constraints available, and a reasonable substitute was used instead. Being upfront about exactly where and why those substitutions happened is more useful to someone extending this project later than quietly pretending they do not exist.

The AI route optimizer is pure Java, not Python and Stable-Baselines3

The original proposal listed Python's Stable-Baselines3 — a real, widely used reinforcement-learning library — as the intended AI backend. Connecting a Java desktop application to a separate Python process is a perfectly reasonable thing to do in a production system, but it introduces a considerable number of additional moving parts: process management, some communication protocol between the two languages, and a separate Python environment that would need to be installed and kept in sync alongside the Java one. None of that serves a first working version particularly well. Instead, QLearningRouteOptimizer is a real, fully functioning Q-learning implementation, written directly in Java with no external machine-learning framework involved at all. It implements the exact same RoutingAlgorithm interface every other algorithm in the project implements — so if a genuine Python and Stable-Baselines3 backend is wanted later, that would mean replacing this one class with an HTTP or gRPC client, and nothing else anywhere in the project would need to change as a result.

No SQLite

The original technology list included SQLite for persistence. Everything this project actually needs to persist — network topologies, performance history, a short list of recently opened files — is naturally small and naturally shaped as either a JSON document or a flat table, so hand-rolled JSON, CSV, and flat-file persistence covers every real need without pulling in a database driver at all. If this project is ever extended with something that genuinely needs relational queries or concurrent multi-user access, that is exactly the point at which reaching for a real database would finally start to pay for itself.

No PDF library

MiniPdf hand-generates simple, genuinely valid PDF files using nothing more than the standard, non-embedded Helvetica font and plain text layout. This approach works well for a straightforward text report; it would not be the right tool if embedded charts, images, or complex multi-column layout inside the PDF itself were ever wanted — at that point, a real library such as Apache PDFBox, the common choice in the Java ecosystem, would be the correct call.

Benchmark Mode compares utilization and delivery counts, not CPU or memory

The original proposal's benchmark comparison table included CPU and memory usage columns. Those genuinely make sense when comparing, for instance, two different physical server machines — but this simulator does not model router hardware in any way at all, so a "CPU usage" figure would have had to be entirely fabricated rather than actually measured from anything real. Those two columns were swapped out for metrics the engine genuinely does track and measure instead.

The packet checksum is real, but currently always valid

Packet.computeChecksum() genuinely does compute a value derived from the packet's real contents — it is not a hardcoded placeholder string. However, there is currently no mechanism anywhere in the simulator that ever actually corrupts a packet's payload; drops in this simulator model congestion, TTL expiry, and link failure, not bit-level data corruption — so the checksum will, at present, always come back valid. This is an honest, deliberately visible placeholder for exactly the kind of feature — simulating an unreliable link genuinely corrupting data in transit — that someone extending this project could meaningfully add next.

Chapter 9 — How to Extend This Project

Some concrete starting points for building on top of NetSimX, arranged roughly from smallest and most approachable to largest and most ambitious.

- **A new traffic type.** Add a new case to TrafficGenerator.TrafficType with its own size range, priority, and protocol. Nothing else in the project needs to change — the QoS scheduler, the statistics collector, and the entire interface already work generically across whatever priority classes happen to exist.
- **A new routing algorithm.** Implement the RoutingAlgorithm interface, which requires exactly one method: given a topology and a source router, return the best path to every other router. Add it to the algorithm dropdown in ControlPanel and to the algorithm checklist in BenchmarkScreen, and it becomes fully usable everywhere else in the application with no further changes required — this is the entire reason that interface exists in the first place.
- **A new topology template.** Add a case to TopologyGenerator.Template and a matching private generator method. It will appear automatically in the New Simulation Wizard's list of radio-button options.
- **Real packet corruption.** Give Packet a corrupted flag, have SimulationEngine roll a chance to set that flag alongside the existing Link.lossProbability check, and make computeChecksum() and isChecksumValid() genuinely reflect the result. The Packet Inspector panel already has a field standing ready to display it.
- **BGP, MPLS, IPv6, or Software-Defined Networking.** All explicitly out of scope for this version of the project — the original proposal's own Future Scope section says as much directly — and all genuinely substantial undertakings in their own right. The project's modular structure is nonetheless deliberately arranged so that none of them would require rewriting the simulation core: a routing algorithm is just one class implementing one interface, and a new protocol is just a new enum value plus behavior added to TcpUdpManager or a sibling class alongside it.
- **Real database persistence.** If flat-file JSON and CSV storage is ever outgrown — for instance, wanting a shared server tracking many different users' simulations at once — persistence.TopologyIO and persistence.CsvExporter are the two specific places to replace; everything upstream of them, the interface and the simulation engine alike, has no awareness of how persistence is actually implemented underneath.
- **A headless or server mode.** BenchmarkRunner already proves conclusively that the simulation engine runs perfectly well with zero graphical interface involvement whatsoever. A command-line tool, a web API, or a batch-processing service built directly on top of SimulationEngine — entirely skipping the gui package — is very achievable without touching the simulation engine itself at all.

Chapter 10 — Glossary

- **Router —** A node in the network that receives packets and forwards them onward toward their eventual destination.
- **Link —** A connection between two routers, carrying a cost, a latency, and a bandwidth.
- **Packet —** One unit of data moving through the network, one hop at a time.
- **TTL (Time To Live) —** A countdown carried by every packet, reduced by one at every hop; if it reaches zero before the packet arrives, the packet is discarded. Prevents packets from circulating forever if something goes wrong with routing.
- **Routing algorithm —** The method used to decide the best path from one router to another. Four are implemented in this project: Dijkstra, Bellman-Ford, ECMP, and a Q-learning-based AI optimizer.
- **Congestion —** What happens when packets arrive at a router faster than it can send them onward; its queue fills up and new arrivals begin being dropped.
- **QoS (Quality of Service) —** Prioritizing some traffic ahead of other traffic when a network is too busy to send everything at once — for instance, sending a voice call's packets before a background file download's.
- **TCP vs UDP —** Two different sets of rules for sending data. TCP guarantees delivery, automatically resending anything that goes missing, at the cost of extra overhead. UDP simply sends and never checks whether the data arrived, which is faster and simpler but can silently lose data. Real voice and video calls typically use UDP, since a late packet is useless anyway; file transfers and web pages typically use TCP, since correctness matters more than raw speed.
- **ACK (Acknowledgement) —** A small packet TCP sends back confirming successful receipt, which is how TCP knows when it is safe to stop waiting and resend something instead.
- **ECMP (Equal-Cost Multi-Path) —** Spreading traffic across several equally good paths instead of always relying on just one, to use available network capacity more evenly.
- **Reinforcement learning / Q-learning —** A machine-learning approach in which an agent learns good decisions purely through trial, error, and observed outcomes, without ever being explicitly programmed with the underlying rules.

Chapter 11 — Closing Notes

Every screenshot appearing in this book, and every specific figure quoted anywhere in Chapters 6 and 7, was captured or measured directly from the real, running application during development — never written from memory, and never aspirational. Wherever this book describes something as verified, an automated check was genuinely carried out, its real output was inspected, and only then was the result written up here.

If there is one single habit worth carrying forward into whatever gets built next on top of this project, it is exactly that one: actually run the thing, and check the real output, rather than trusting that a piece of code should probably work because it reads correctly on the page. That habit is, in the most literal sense possible, the reason the two bugs documented in Chapter 7 were ever found and fixed at all — and it is very likely to keep paying off in whatever comes next.

Good luck with wherever you take this next.